

POC A — React Web Speech Pipeline

Replace Web Speech with our own capture path and phoneme-level scoring, then measure it

Owner: Harshan **Date:** 28 July 2026 **Duration:** 10 working days **Status:** Ready to assign

1. What this POC answers

Three questions, in priority order:

#	Question	Why it matters
Q1	Does phoneme-level scoring separate accented-but-correct speech from fluent-but-incorrect speech?	If not, no platform helps. Everything downstream depends on this.
Q2	Does our current live feature separate them?	Tells us the size of the problem we may already have in production
Q3	Do iPhone, iPad, Mac and desktop produce comparable scores?	Determines whether POC B is needed at all

This POC is also the control condition for POC B. Same fixture, same scorer, same output format — only the capture path differs. That is what makes the comparison meaningful, and it is why POC A must complete first.

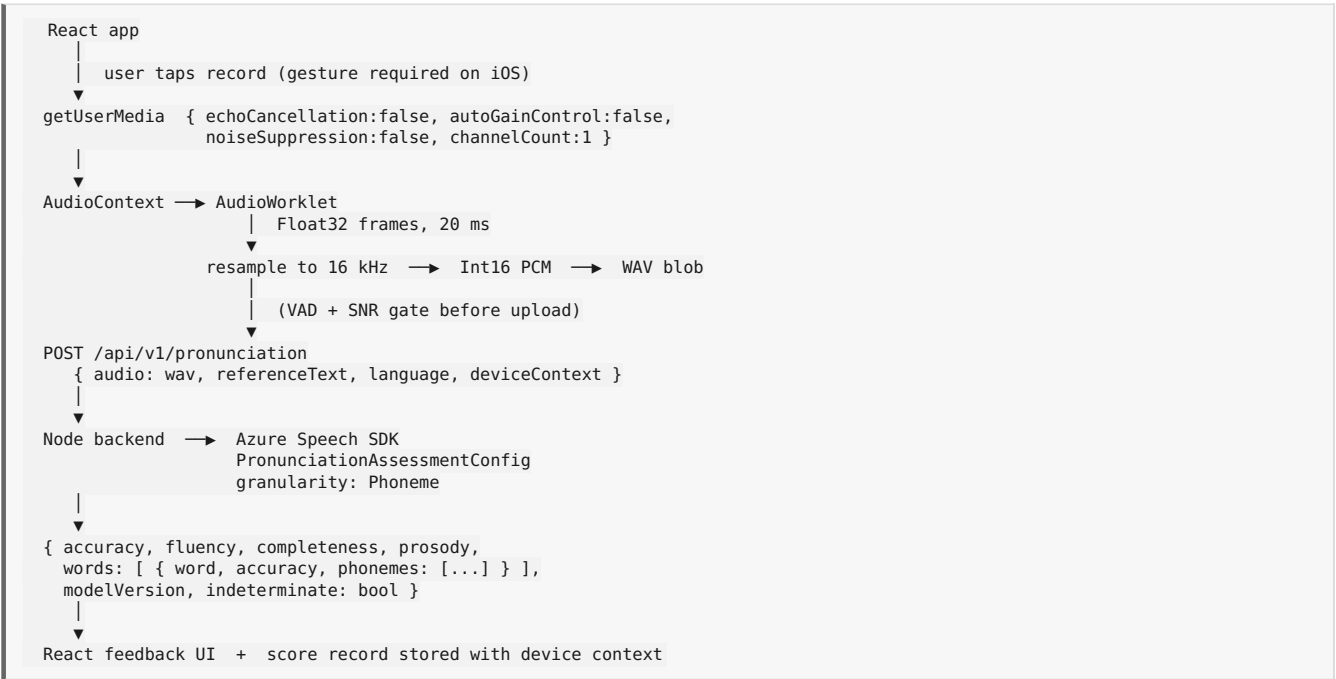
2. Success criteria

#	Criterion
S1	A learner records a target phrase in the browser and receives phoneme-level feedback
S2	Works on iPhone Safari, iPad Safari, Mac Safari, Mac Chrome, and Android Chrome
S3	No Web Speech API anywhere in the POC code path
S4	Fixture of 80 recordings scored through both the new path and the current live feature
S5	Score distributions plotted per set, per platform, with a written verdict
S6	Every capture session emits telemetry: device, browser, constraints granted, timings, errors

3. Out of scope

Streaming recognition. Free-speech dictation. WebSocket transport. Multi-provider routing. Production UI. Auth beyond what already exists. Offline scoring. Anything in the Cochlea document set beyond what is listed below.

4. Architecture



Note: batch upload, not streaming. Scoring a bounded utterance against a known reference text requires the complete recording

- streaming adds complexity and reduces nothing.

5. Work packages

Roles: **FE** frontend · **BE** backend · **QA** test · **CONT** content/fixture · **LEAD** Harshan

Language split. The endpoint (A7-A10) is **Node**, added to the existing Lingotran backend — no second runtime, no second deploy target on App Service. The fixture runner and analysis (A14, A17) are **Python** scripts run locally or in CI, because `pandas`, `soundfile` and `matplotlib` have no real Node equivalent. Different lifecycles, no coupling: nobody operates the Python side.

ID	Task	Role	Days	Depends on
A1	Deploy <code>probe.html</code> to an HTTPS host; run across all five target devices; record constraint reports and DSP measurements	QA	1	—
A2	Instrument the current live feature : log success/failure, browser, OS, error reason per attempt	FE	1	—
A3	AudioWorklet capture module — worklet processor, resampler, level meter, frame emit	FE	2	—
A4	Capture wrapper — permission state machine, device change detection, suspended-context handling, gesture entry point, typed error codes	FE	2	A3
A5	Client VAD + SNR gate — reject or warn on silence and excessive noise before upload	FE	1.5	A3
A6	WAV encoding and upload with retry	FE	0.5	A3
A7	Node endpoint <code>POST /api/v1/pronunciation</code> — accept audio + reference text, validate, forward	BE	1	—
A8	Azure Speech SDK integration — pronunciation assessment, phoneme granularity, credentials in Key Vault	BE	1.5	A7
A9	Score normalization — one response shape, <code>indeterminate</code> below confidence floor, record model version	BE	1	A8
A10	Telemetry — persist device, browser, constraints granted, timings, errors, scores per attempt	BE	1	A7
A11	Minimal feedback UI — record button, level meter, per-word and per-phoneme result, "unclear, try again" state	FE	1.5	A4, A9
A12	Record fixture Set A — 40 target words pronounced <i>acceptably</i> by speakers with strong L1 accents, validated by a fluent rater	CONT	3	—
A13	Record fixture Set B — same words <i>deliberately mispronounced</i> by fluent speakers, realistic error types	CONT	2	A12
A14	Fixture runner and analysis tooling — batch-submit both sets, capture results, export CSV, plot distributions. Python (<code>pandas</code> , <code>soundfile</code> , <code>matplotlib</code>); scripts, not a deployed service.	BE	1	A9, A13
A15	Run fixture through the current live feature for comparison	QA	1	A2, A13
A16	Device matrix run — record the fixture natively on each of the five targets, score, export	QA	2	A11, A13
A17	Analysis — plot distributions per set per platform, compute separation, write verdict	LEAD	1.5	A14, A15, A16

Total effort: ~24 person-days across 5 roles · **Calendar:** 10 working days with parallel tracks

6. Acceptance criteria per package

ID	Accepted when
A1	Probe JSON captured for all five devices; a Mac Chrome control reading confirms the method
A3	Continuous 16 kHz mono Int16 frames, gap-free, verified against a reference tone
A4	Every error code in the taxonomy is reachable and produces actionable UI text; muted input raises an error within 3 seconds
A5	Silence-only input is rejected before upload; a noisy recording produces a warning, not a score
A8	Phoneme-level scores returned for a known-good recording; no credentials reachable from the browser
A9	Response shape identical regardless of input; <code>indeterminate</code> returned below the floor rather than a number
A11	End-to-end on iPhone Safari from a cold load: tap, speak, see per-phoneme feedback
A12	Every Set A recording independently rated <i>acceptable</i> by a fluent speaker; ambiguous ones dropped
A13	Errors are realistic learner mistakes — substituted phonemes, wrong stress, dropped final consonants — not comic mispronunciation
A17	A written verdict mapping to one of the four outcomes in §8

7. Sequencing

Day	1	2	3	4	5	6	7	8	9	10	
QA	A1	_____						A15	A16	A16	—
FE	A3	A3	A4	A4	A5	A5	A6	A11	A11	—	
BE	A7	A8	A8	A9	A10	—		A14	_____		
CONT	A12	A12	A12	A13	A13	_____					
LEAD	_____								A17	A17	

Start A12 on day one. Fixture recording is the critical path — it needs people, scheduling, and rating, and everything in §8 waits on it.

A1 and A2 also start day one. Both are cheap and either may resolve the reliability question before the build finishes.

8. The decision this POC produces

Outcome	Meaning	Next action
Sets separate cleanly on all platforms	Scoring works; iOS processing is not a factor	Ship on web. Cancel POC B.
Sets separate; iPhone consistently offset	Real but deterministic platform bias	Ship on web with per-platform calibration. POC B optional — run only if the offset exceeds tolerance.
Sets separate on desktop; unreliable on iPhone	Genuine platform problem	Proceed to POC B with evidence
Sets do not separate anywhere	The scorer is not the answer	Stop both. Investigate capture format, reference text handling, and language configuration before building further.

POC B is gated on this result. Do not start its native work before A17 is complete, other than the shell and plugin scaffolding noted in that plan.

9. Risks

Risk	Mitigation
Set A recordings are not genuinely acceptable pronunciation	Independent rating by a fluent speaker; drop anything ambiguous. A contaminated Set A invalidates the entire experiment.
Fixture recruiting slips	Start day one; 30 usable recordings per set is enough to see separation if it exists
iOS Safari ignores capture constraints	Expected. A1 measures it; this is an input to the analysis, not a blocker
Azure audio format requirements differ from our output	Verify format requirements against current docs before A3 begins
Current live feature is harder to instrument than expected	A15 can fall back to manual spot-checks on 20 recordings
Azure quota or region latency	Confirm tier and region before A8

10. Deliverables

- Working web capture and scoring path, deployed to a test environment
- Probe results across five devices
- Fixture: 80 validated recordings, versioned and stored
- Fixture results: new path, current feature, per platform, as CSV
- Analysis document with distribution plots and a verdict mapped to §8
- Telemetry baseline for the current live feature

11. Results template

Fill this in at A17. Numbers here are placeholders.

Platform	Set A median	Set B median	Overlap	Verdict
Mac Chrome (control)	—	—	—	—
Mac Safari	—	—	—	—
iPhone Safari	—	—	—	—
iPad Safari	—	—	—	—
Android Chrome	—	—	—	—
Current live feature	—	—	—	—

Separation is the gap between Set A's lower quartile and Set B's upper quartile. Positive means the scorer distinguishes them. Negative means it does not.